TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN

BÀI THỰC HÀNH 3

NEXT.JS — DATA FETCHING, API ROUTES & UI

Môn học: Các công nghệ mới trong phát triển phần mềm
Thời lượng: 4 tiết (180 phút)

Đà Lạt, 2026

MỤC LỤC

Mục tiêu bài thực hành 1

Yêu cầu đầu vào 1

Quy ước ký hiệu 1

Giới thiệu: Nâng cấp website Portfolio/Blog 1

Phần 1: Data Fetching — Server-side 2

- **1.1. Các patterns Data Fetching trong Next.JS** 2
- **1.2. JSONPlaceholder — API giả lập miễn phí** 3
- **1.3. Thực hành: Định nghĩa TypeScript types** 3
- **1.4. Thực hành: Fetch danh sách bài viết từ API** 4
- **1.5. Thực hành: Cập nhật Dynamic Route cho trang chi tiết** 6
- **1.6. Caching & Revalidation** 8
- **1.7. Commit thay đổi** 9
- **Bài tập tự làm — Phần 1** 9

Phần 2: API Routes (Route Handlers) 10

- **2.1. Route Handlers là gì?** 10
- **2.2. Tạo data store tạm thời (in-memory)** 11
- **2.3. Tạo API Route — GET (lấy danh sách)** 12
- **2.4. Tạo API Route — POST (thêm lời nhắn mới)** 12
- **2.5. Tạo API Route — DELETE (xóa lời nhắn)** 14
- **2.6. Kiểm tra tổng hợp API** 15
- **2.7. Commit thay đổi** 15
- **Bài tập tự làm — Phần 2** 15

Phần 3: Client-side Data Fetching 15

- **3.1. Khi nào cần Client-side fetch?** 16
- **3.2. Thực hành: Tạo trang Guestbook** 16
- **3.3. Client-side fetch: Hạn chế và cải tiến** 21
- **3.4. Commit thay đổi** 22
- **Bài tập tự làm — Phần 3** 22

Phần 4: Server Actions & Form Handling 22

- **4.1. Server Actions là gì?** 23
- **4.2. Cài đặt Zod — Schema Validation** 23
- **4.3. Thực hành: Server Action cho form Guestbook** 24
- **4.4. Thực hành: Tạo form component với Server Actions** 26
- **4.5. Thực hành: Cải tiến form Liên hệ** 30

4.6. Commit thay đổi 34
Bài tập tự làm — Phần 4 34
Phần 5: shadcn/ui — Component Library 35
5.1. shadcn/ui là gì? 35
5.2. Cài đặt shadcn/ui 35
5.3. Cài đặt các components cần thiết 36
5.4. Sử dụng Button component 37
5.5. Sử dụng Card component 37
5.6. Thực hành: Nâng cấp trang Blog với shadcn/ui 38
5.7. Thực hành: Nâng cấp form Guestbook với shadcn/ui 39
5.8. Thực hành: Nâng cấp trang Projects với Card và Badge 41
5.9. Kiểm tra kết quả 42
5.10. Commit thay đổi 43
Bài tập tự làm — Phần 5 43
Phần 6: Tổng kết & Yêu cầu nộp bài 43
Tóm tắt kiến thức 44
Checklist hoàn thành 44
Bài tập nộp điểm 45
Chuẩn bị cho buổi sau 46
Tài liệu tham khảo 46

Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

1. Hiểu và vận dụng các patterns Data Fetching trong Next.JS (server-side, client-side)
2. Tạo API Routes (Route Handlers) xử lý các thao tác CRUD cơ bản
3. Sử dụng Server Actions để xử lý form submission hiện đại
4. Validate dữ liệu đầu vào bằng Zod schema validation
5. Cài đặt và sử dụng shadcn/ui component library để xây dựng giao diện chuyên nghiệp

Yêu cầu đầu vào

Kiến thức nền:

- Hoàn thành Bài thực hành 2 (đã có website portfolio/blog với routing, layouts, Tailwind CSS)
- Kiến thức React cơ bản: component, props, state, hooks (useState, useEffect)
- TypeScript cơ bản: interface, types
- Hiểu HTTP methods: GET, POST, PUT, DELETE

Phần cứng & phần mềm:

- Máy tính đã cài đặt Node.js, VS Code, Git (từ Bài TH 1)
- Project `ctk46-lab01` đã push lên GitHub (từ Bài TH 2)

Quy ước ký hiệu

Ký hiệu	Ý nghĩa
	Mẹo hữu ích
	Lưu ý quan trọng
	Bài tập tự làm
	Giải thích thêm
	Kết quả mong đợi

Giới thiệu: Nâng cấp website Portfolio/Blog

Trong bài thực hành này, chúng ta tiếp tục phát triển website portfolio/blog từ Bài TH 2. Hiện tại, website đang sử dụng dữ liệu cứng (hardcoded) trong file `src/data/posts.ts`. Hôm nay chúng ta sẽ:

Tính năng mới	Mô tả
Data Fetching	Thay thế dữ liệu cứng bằng cách gọi API thực tế
API Routes	Tạo backend API cho tính năng Sổ lưu bút (Guestbook)
Server Actions	Xử lý form liên hệ và guestbook hiện đại
Zod Validation	Kiểm tra dữ liệu đầu vào an toàn
shadcn/ui	Nâng cấp giao diện với component library chuyên nghiệp

Khi hoàn thành, website sẽ có thêm trang **Sổ lưu bút** (`/guestbook`) cho phép khách truy cập để lại lời nhắn, và toàn bộ giao diện được nâng cấp với shadcn/ui.

Phần 1: Data Fetching — Server-side

Thời lượng: *Thời lượng: ~30 phút*

Trong phần này, chúng ta sẽ học cách lấy dữ liệu từ API bên ngoài trong Next.JS, thay thế dữ liệu cứng (hardcoded) đang dùng ở Lab 2.

1.1. Các patterns Data Fetching trong Next.JS

Next.JS hỗ trợ nhiều cách lấy dữ liệu (data fetching), mỗi cách phù hợp với tình huống khác nhau:

Pattern	Nơi thực thi	Cách dùng	Khi nào dùng
Server-side fetch	Server	`fetch()` trực tiếp trong Server Component	Dữ liệu từ database, API — cần SEO, bảo mật
Client-side fetch	Browser	`useEffect` + `fetch()`	Dữ liệu real-time, tương tác người dùng
SWR / React Query	Browser	Hook library	Client-side fetch nâng cao (cache, revalidate)
Server Actions	Server	`"use server"`	Xử lý form, mutations

		functions	(thêm/sửa/xóa)

Giải thích: Điểm mấu chốt: Trong Next.JS App Router, **tất cả components đều là Server Components theo mặc định**. Điều này có nghĩa bạn có thể gọi `fetch()` hoặc truy vấn database trực tiếp trong component mà không cần `useEffect` hay `useState`. Đây là điểm khác biệt lớn so với React thuần.

1.2. JSONPlaceholder — API giả lập miễn phí

Trong bài thực hành này, chúng ta sẽ sử dụng JSONPlaceholder (https://jsonplaceholder.typicode.com) — một API giả lập miễn phí dành cho việc học tập và prototype.

JSONPlaceholder cung cấp các endpoint sau:

Endpoint	Mô tả	Số lượng
`/posts`	Bài viết blog	100
`/users`	Người dùng	10
`/comments`	Bình luận	500
`/photos`	Hình ảnh	5000

Ví dụ, truy cập `https://jsonplaceholder.typicode.com/posts/1` sẽ trả về:

```Json
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio
reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae..."
}
```

1.3. Thực hành: Định nghĩa TypeScript types

Trước khi fetch data, ta cần định nghĩa kiểu dữ liệu TypeScript phù hợp với cấu trúc API.

Mở project `ctk46-lab01` trong VS Code:

```Bash
cd ctk46-lab01
code .
```

Tạo file `src/types/post.ts`:

```bash
mkdir -p src/types
```

```tsx
export interface Post {
  userId: number;
  id: number;
  title: string;
  body: string;
}

export interface User {
  id: number;
  name: string;
  username: string;
  email: string;
  phone: string;
  website: string;
  company: {
    name: string;
    catchPhrase: string;
  };
}
```

Giải thích: Giải thích: Các interface này phản ánh chính xác cấu trúc JSON trả về từ JSONPlaceholder API. TypeScript sẽ giúp ta được gợi ý code (IntelliSense) và phát hiện lỗi sớm khi sử dụng dữ liệu.

1.4. Thực hành: Fetch danh sách bài viết từ API

Bây giờ, thay vì import dữ liệu cứng từ `src/data/posts.ts`, chúng ta sẽ gọi API để lấy dữ liệu thực.

Cập nhật file `src/app/blog/page.tsx`:

```tsx
import Link from "next/link";
import { Post } from "@/types/post";

async function getPosts(): Promise<Post[]> {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");

  if (!res.ok) {
    throw new Error("Không thể tải danh sách bài viết");
  }

  return res.json();
}
```

```
export default async function BlogPage() {
  const posts = await getPosts();

  return (
    <div>
      <h1 className="text-3xl font-bold mb-2">Blog</h1>
      <p className="text-gray-500 mb-6">
        Tổng cộng {posts.length} bài viết từ API
      </p>

      <div className="space-y-6">
        {posts.slice(0, 10).map((post) => (
          <article
            key={post.id}
            className="border rounded-lg p-6 hover:shadow-md transition-shadow"
          >
            <div className="flex items-center gap-3 mb-2">
              <span className="bg-blue-100 text-blue-700 text-xs px-2 py-1
rounded">
                Tác giả #{post.userId}
              </span>
              <span className="text-sm text-gray-400">Bài #{post.id}</span>
            </div>

            <Link href={`/blog/${post.id}`}>
              <h2 className="text-xl font-semibold mb-2 hover:text-blue-600
transition-colors capitalize">
                {post.title}
              </h2>
            </Link>

            <p className="text-gray-600 line-clamp-2">{post.body}</p>

            <Link
              href={`/blog/${post.id}`}
              className="inline-block mt-3 text-blue-600 text-sm hover:underline"
            >
              Đọc thêm →
            </Link>
          </article>
        ))}
      </div>
    </div>
  );
}
```

Giải thích: Giải thích chi tiết:

- `async function getPosts()` — Hàm bất đồng bộ (async) gọi API. Vì đây là Server Component, ta có thể gọi `fetch()` trực tiếp mà không cần `useEffect`.

- `export default async function BlogPage()` — Server Component có thể là `async` function. Đây là điểm khác biệt so với Client Component (không thể là async).
- `const posts = await getPosts()` — Chờ dữ liệu trước khi render. Next.JS sẽ tự động hiển thị `loading.tsx` (đã tạo ở Lab 2) trong khi chờ.
- `posts.slice(0, 10)` — Chỉ hiển thị 10 bài đầu tiên (API trả về 100 bài).
- `throw new Error(...)` — Nếu API lỗi, error sẽ được bắt bởi `error.tsx` (đã tạo ở Lab 2).

Lưu ý: Lưu ý: Dynamic route blog bây giờ sử dụng `id` (số) thay vì `slug` (chuỗi). Ta cần cập nhật thư mục `[slug]` thành `[id]`.

1.5. Thực hành: Cập nhật Dynamic Route cho trang chi tiết

Đổi tên thư mục `src/app/blog/[slug]` thành `src/app/blog/[id]`:

```bash
mv src/app/blog/\[slug\] src/app/blog/\[id\]
```

Cập nhật file `src/app/blog/[id]/page.tsx`:

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { Post, User } from "@/types/post";

interface BlogPostPageProps {
  params: Promise<{ id: string }>;
}

async function getPost(id: string): Promise<Post> {
  const res = await fetch(
    `https://jsonplaceholder.typicode.com/posts/${id}`
  );

  if (!res.ok) {
    notFound();
  }

  return res.json();
}

async function getUser(userId: number): Promise<User> {
  const res = await fetch(
    `https://jsonplaceholder.typicode.com/users/${userId}`
  );
```

```
  if (!res.ok) {
    throw new Error("Không thể tải thông tin tác giả");
  }

  return res.json();
}

export default async function BlogPostPage({ params }: BlogPostPageProps) {
  const { id } = await params;
  const post = await getPost(id);
  const author = await getUser(post.userId);

  return (
    <div>
      <Link
        href="/blog"
        className="text-blue-600 hover:underline text-sm mb-6 inline-block"
      >
        ← Quay lại danh sách
      </Link>

      <article>
        <h1 className="text-3xl font-bold mb-4 capitalize">{post.title}</h1>

        <div className="flex items-center gap-3 mb-6 text-sm text-gray-500">
          <span>Tác giả: <strong className="text-gray-
700">{author.name}</strong></span>
          <span>•</span>
          <span>{author.email}</span>
        </div>

        <div className="prose max-w-none text-gray-700 whitespace-pre-line mb-8
leading-relaxed">
          {post.body}
        </div>

        <div className="border-t pt-6">
          <h3 className="font-semibold mb-2">Về tác giả</h3>
          <p className="text-gray-600 text-sm">
            <strong>{author.name}</strong> (@{author.username}) —
{author.company.name}
          </p>
          <p className="text-gray-500 text-sm">{author.company.catchPhrase}</p>
        </div>
      </article>
    </div>
  );
}
```

Giải thích: Giải thích:

- Hàm `getPost()` và `getUser()` — Hai hàm fetch riêng biệt, mỗi hàm gọi một endpoint khác nhau.

- `const post = await getPost(id)` rồi `const author = await getUser(post.userId)` — Hai request chạy tuần tự vì request thứ hai cần `userId` từ request đầu tiên.
- `notFound()` — Function từ `next/navigation`, khi gọi sẽ hiển thị trang `not-found.tsx` (đã tạo ở Lab 2).

Cập nhật file `src/app/blog/[id]/not-found.tsx` (nếu tên thư mục đã đổi):

```tsx
import Link from "next/link";

export default function BlogNotFound() {
  return (
    <div className="text-center py-16">
      <h2 className="text-2xl font-bold mb-4">Bài viết không tồn tại</h2>
      <p className="text-gray-600 mb-6">
        Bài viết bạn tìm kiếm không tồn tại hoặc đã bị xóa.
      </p>
      <Link href="/blog" className="text-blue-600 hover:underline">
        ← Quay lại danh sách bài viết
      </Link>
    </div>
  );
}
```

Chạy development server và kiểm tra:

```bash
npm run dev
```

Kết quả mong đợi: Kết quả mong đợi:

- Truy cập `http://localhost:3000/blog` — hiển thị 10 bài viết từ JSONPlaceholder API
- Truy cập `http://localhost:3000/blog/1` — hiển thị chi tiết bài viết #1 với thông tin tác giả
- Truy cập `http://localhost:3000/blog/999` — hiển thị trang "Bài viết không tồn tại"

1.6. Caching & Revalidation

Mặc định, Next.JS sẽ cache kết quả `fetch()` trong Server Components. Điều này có nghĩa dữ liệu chỉ được tải một lần và sử dụng lại cho các request tiếp theo.

Có 3 chiến lược caching phổ biến:

Chiến lược	Cách cấu hình	Ý nghĩa
Static (mặc định)	`fetch(url)`	Cache vĩnh viễn, build 1 lần
Time-based revalidation	`fetch(url, { next: { revalidate: 60 } })`	Cache trong 60 giây, sau đó tải lại
No cache	`fetch(url, { cache: 'no-store' })`	Luôn tải mới mỗi request

Ví dụ, nếu muốn trang blog tự động cập nhật mỗi 60 giây:

```tsx
async function getPosts(): Promise<Post[]> {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    next: { revalidate: 60 },
  });

  if (!res.ok) {
    throw new Error("Không thể tải danh sách bài viết");
  }

  return res.json();
}
```

Mẹo: Với `revalidate: 60`, Next.JS sẽ phục vụ trang từ cache trong 60 giây, sau đó tự động gọi lại API ở lần request tiếp theo. Đây chính là **ISR (Incremental Static Regeneration)** — kết hợp ưu điểm của static (nhanh) và dynamic (cập nhật).

1.7. Commit thay đổi

```bash
git add .
git commit -m "feat: thay thế data cứng bằng fetch từ JSONPlaceholder API"
```

Bài tập tự làm — Phần 1

1. Thử thay `fetch()` bằng `fetch(url, { cache: 'no-store' })` trong hàm `getPosts()`, reload trang nhiều lần và quan sát sự khác biệt (kiểm tra tab Network trong DevTools)
2. Trong trang chi tiết bài viết (`/blog/[id]`), thêm phần hiển thị danh sách comments của bài viết bằng cách gọi thêm API: `https://jsonplaceholder.typicode.com/posts/{id}/comments`

3. Tìm cách chạy 2 request `getPost()` và `getUser()` song song bằng `Promise.all()` thay vì tuần tự. Tại sao điều này tốt hơn?

Phần 2: API Routes (Route Handlers)

Thời lượng: *Thời lượng: ~35 phút*

Trong phần này, chúng ta sẽ tạo backend API ngay trong Next.JS bằng **Route Handlers** — cho phép xử lý các HTTP request (GET, POST, DELETE) mà không cần server riêng.

2.1. Route Handlers là gì?

Route Handlers cho phép tạo API endpoints trong Next.JS bằng cách sử dụng file `route.ts` trong thư mục `src/app/api/`. Cách hoạt động tương tự như API của Express.js hoặc các backend framework khác.

Quy tắc:

```
File                                        URL tương ứng
────────────────────────                    ─────────────────

src/app/api/hello/route.ts                → /api/hello
src/app/api/guestbook/route.ts            → /api/guestbook
src/app/api/guestbook/[id]/route.ts       → /api/guestbook/123
```

Cấu trúc cơ bản của Route Handler:

```tsx
import { NextRequest, NextResponse } from "next/server";

// GET /api/endpoint
export async function GET(request: NextRequest) {
  return NextResponse.json({ message: "Hello" });
}

// POST /api/endpoint
export async function POST(request: NextRequest) {
  const body = await request.json();
  return NextResponse.json({ data: body }, { status: 201 });
}
```

Giải thích: Giải thích:

- Mỗi HTTP method (GET, POST, PUT, DELETE) được export dưới dạng **named function**.

- `NextRequest` — Object chứa thông tin request (headers, body, query params, ...).
- `NextResponse.json()` — Trả về JSON response với status code tùy chọn.

Lưu ý: Quan trọng: File `route.ts` và file `page.tsx` **không thể tồn tại trong cùng một thư mục**. Nếu bạn đã có `src/app/guestbook/page.tsx`, API phải đặt ở `src/app/api/guestbook/route.ts`.

2.2. Tạo data store tạm thời (in-memory)

Trước khi tạo API, ta cần nơi lưu trữ dữ liệu. Ở Lab 4 (Supabase), ta sẽ dùng database thực. Hiện tại, ta sẽ lưu trong biến JavaScript (in-memory) — dữ liệu sẽ mất khi restart server.

Tạo file `src/data/guestbook.ts`:

```tsx
export interface GuestbookEntry {
  id: string;
  name: string;
  message: string;
  createdAt: string;
}

export const guestbookEntries: GuestbookEntry[] = [
  {
    id: "1",
    name: "Nguyễn Văn B",
    message: "Website đẹp quá! Chúc bạn thành công nhé!",
    createdAt: new Date("2025-03-01").toISOString(),
  },
  {
    id: "2",
    name: "Trần Thị C",
    message: "Rất ấn tượng với portfolio của bạn. Keep going!",
    createdAt: new Date("2025-03-05").toISOString(),
  },
  {
    id: "3",
    name: "Lê Văn D",
    message: "Mình cũng đang học Next.JS, mong được giao lưu!",
    createdAt: new Date("2025-03-10").toISOString(),
  },
];
```

Giải thích: Giải thích:

- `GuestbookEntry` — Interface định nghĩa cấu trúc mỗi lời nhắn: id, tên người gửi, nội dung, thời gian tạo.

- `guestbookEntries` — Mảng lưu dữ liệu, được export trực tiếp (mutable). Các API route sẽ thao tác trực tiếp trên mảng này.
- Dữ liệu mẫu giúp ta có sẵn vài lời nhắn khi test.

Lưu ý: Lưu ý: Vì dữ liệu lưu trong memory, mỗi khi restart dev server (`npm run dev`), dữ liệu sẽ quay về trạng thái ban đầu. Đây là hành vi có chủ đích — ở Lab 4 ta sẽ dùng Supabase database để lưu trữ lâu dài.

2.3. Tạo API Route — GET (lấy danh sách)

Tạo thư mục và file:

```bash
mkdir -p src/app/api/guestbook
```

Tạo file `src/app/api/guestbook/route.ts`:

```tsx
import { NextRequest, NextResponse } from "next/server";
import { guestbookEntries } from "@/data/guestbook";

// GET /api/guestbook — Lấy danh sách tất cả lời nhắn
export async function GET() {
  return NextResponse.json(guestbookEntries);
}
```

Chạy dev server và test:

```bash
npm run dev
```

Mở trình duyệt, truy cập: `http://localhost:3000/api/guestbook`

Kết quả mong đợi: Kết quả mong đợi: Trình duyệt hiển thị JSON chứa 3 lời nhắn mẫu.

Mẹo: Bạn cũng có thể test bằng terminal:

```bash
curl http://localhost:3000/api/guestbook
```

2.4. Tạo API Route — POST (thêm lời nhắn mới)

Thêm function `POST` vào file `src/app/api/guestbook/route.ts`:

```tsx
import { NextRequest, NextResponse } from "next/server";
import { guestbookEntries } from "@/data/guestbook";

// GET /api/guestbook — Lấy danh sách tất cả lời nhắn
export async function GET() {
  return NextResponse.json(guestbookEntries);
}

// POST /api/guestbook — Thêm lời nhắn mới
export async function POST(request: NextRequest) {
  const body = await request.json();

  // Kiểm tra dữ liệu đầu vào
  if (!body.name || !body.message) {
    return NextResponse.json(
      { error: "Tên và lời nhắn là bắt buộc" },
      { status: 400 }
    );
  }

  // Tạo entry mới
  const newEntry = {
    id: Date.now().toString(),
    name: body.name,
    message: body.message,
    createdAt: new Date().toISOString(),
  };

  // Thêm vào đầu mảng (hiển thị mới nhất trước)
  guestbookEntries.unshift(newEntry);

  return NextResponse.json(newEntry, { status: 201 });
}
```

Test bằng curl:

```bash
curl -X POST http://localhost:3000/api/guestbook \
  -H "Content-Type: application/json" \
  -d '{"name": "Test User", "message": "Hello from curl!"}'
```

Kết quả mong đợi: Kết quả mong đợi: Trả về JSON của entry vừa tạo với status 201.

Kiểm tra lại danh sách:

```bash
curl http://localhost:3000/api/guestbook
```

Kết quả mong đợi: Kết quả mong đợi: Danh sách bây giờ có 4 entries (entry mới ở đầu).

2.5. Tạo API Route — DELETE (xóa lời nhắn)

Tạo file `src/app/api/guestbook/[id]/route.ts` (dynamic API route):

```bash
mkdir -p src/app/api/guestbook/\[id\]
```

```tsx
import { NextRequest, NextResponse } from "next/server";
import { guestbookEntries } from "@/data/guestbook";

interface RouteParams {
  params: Promise<{ id: string }>;
}

// DELETE /api/guestbook/[id] — Xóa lời nhắn theo id
export async function DELETE(request: NextRequest, { params }: RouteParams) {
  const { id } = await params;

  const index = guestbookEntries.findIndex((entry) => entry.id === id);

  if (index === -1) {
    return NextResponse.json(
      { error: "Không tìm thấy lời nhắn" },
      { status: 404 }
    );
  }

  const deleted = guestbookEntries.splice(index, 1)[0];

  return NextResponse.json(deleted);
}
```

Giải thích: Giải thích:

- `[id]` trong tên thư mục — Dynamic route parameter, tương tự `[slug]` ở Lab 2.
- `params: Promise<{ id: string }>` — Trong Next.JS phiên bản mới, params là Promise cần await.
- `guestbookEntries.splice(index, 1)` — Xóa 1 phần tử tại vị trí `index` và trả về phần tử đã xóa.

Test bằng curl:

```bash
# Xóa entry có id = "1"
curl -X DELETE http://localhost:3000/api/guestbook/1
```

Kết quả mong đợi: Kết quả mong đợi: Trả về JSON của entry đã xóa.

2.6. Kiểm tra tổng hợp API

Thực hiện tuần tự các lệnh sau để kiểm tra toàn bộ luồng CRUD:

```bash
# 1. Xem danh sách (3 entries mẫu)
curl http://localhost:3000/api/guestbook | python3 -m json.tool

# 2. Thêm mới
curl -X POST http://localhost:3000/api/guestbook \
  -H "Content-Type: application/json" \
  -d '{"name": "Sinh viên A", "message": "Bài lab rất bổ ích!"}'

# 3. Xem lại danh sách (4 entries)
curl http://localhost:3000/api/guestbook | python3 -m json.tool

# 4. Xóa entry đầu tiên (id vừa tạo — copy từ kết quả bước 2)
curl -X DELETE http://localhost:3000/api/guestbook/<ID_VUA_TAO>

# 5. Xem lại danh sách (3 entries)
curl http://localhost:3000/api/guestbook | python3 -m json.tool
```

Mẹo: Nếu sử dụng VS Code, bạn có thể cài extension **REST Client** (tên: `humao.rest-client`) để test API ngay trong editor thay vì dùng curl.

2.7. Commit thay đổi

```bash
git add .
git commit -m "feat: tạo API Routes cho Guestbook (GET, POST, DELETE)"
```

Bài tập tự làm — Phần 2

1. Thêm API route **PUT** `/api/guestbook/[id]` để chỉnh sửa nội dung lời nhắn (cập nhật `name` và `message`)
2. Thêm validation trong POST: `name` phải từ 2-50 ký tự, `message` phải từ 1-500 ký tự. Trả về error message cụ thể khi vi phạm
3. Thêm query parameter cho GET endpoint: `GET /api/guestbook?limit=5` chỉ trả về 5 entries gần nhất

Phần 3: Client-side Data Fetching

Thời lượng: *Thời lượng: ~25 phút*

Trong phần này, chúng ta sẽ tạo trang Guestbook trên giao diện, sử dụng **client-side fetch** để gọi API routes vừa tạo ở Phần 2.

3.1. Khi nào cần Client-side fetch?

Nên dùng Server-side fetch	Nên dùng Client-side fetch
Dữ liệu cần SEO (Google index được)	Dữ liệu cập nhật real-time
Dữ liệu không thay đổi theo user	Dữ liệu thay đổi theo tương tác người dùng
Cần truy cập database trực tiếp	Sau khi user submit form, cần refresh data
Bảo mật API keys	Polling, WebSocket

Trong trường hợp trang Guestbook, ta dùng **client-side fetch** vì:

- Người dùng gửi lời nhắn → cần cập nhật danh sách ngay lập tức
- Dữ liệu guestbook không cần SEO
- Cần tương tác (thêm, xóa) mà không reload trang

3.2. Thực hành: Tạo trang Guestbook

Trước tiên, thêm link Guestbook vào Navbar. Cập nhật file `src/components/navbar.tsx`:

```tsx
import Link from "next/link";

export default function Navbar() {
  return (
    <nav className="bg-white shadow-sm border-b">
      <div className="max-w-5xl mx-auto px-4 py-4">
        <div className="flex items-center justify-between">
          <Link href="/" className="text-xl font-bold text-blue-600">
            Portfolio
          </Link>
          <div className="flex gap-6">
            <Link
              href="/"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Trang chủ
            </Link>
            <Link
              href="/about"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Giới thiệu
            </Link>
            <Link
```

```
              href="/blog"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Blog
            </Link>
            <Link
              href="/projects"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Dự án
            </Link>
            <Link
              href="/guestbook"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Lưu bút
            </Link>
            <Link
              href="/contact"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Liên hệ
            </Link>
          </div>
        </div>
      </div>
    </nav>
  );
}
```

Bây giờ tạo trang Guestbook. Tạo thư mục và file:

```bash
mkdir -p src/app/guestbook
```

Tạo file `src/app/guestbook/page.tsx`:

```tsx
"use client";

import { useState, useEffect } from "react";
import { GuestbookEntry } from "@/data/guestbook";

export default function GuestbookPage() {
  const [entries, setEntries] = useState<GuestbookEntry[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // State cho form
  const [name, setName] = useState("");
  const [message, setMessage] = useState("");
  const [submitting, setSubmitting] = useState(false);

  // Fetch danh sách lời nhắn
```

```
  async function fetchEntries() {
    try {
      const res = await fetch("/api/guestbook");
      if (!res.ok) throw new Error("Lỗi khi tải dữ liệu");
      const data = await res.json();
      setEntries(data);
    } catch (err) {
      setError("Không thể tải sổ lưu bút. Vui lòng thử lại.");
    } finally {
      setLoading(false);
    }
  }

  // Gọi fetchEntries khi component mount
  useEffect(() => {
    fetchEntries();
  }, []);

  // Xử lý gửi lời nhắn mới
  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();

    if (!name.trim() || !message.trim()) return;

    setSubmitting(true);
    try {
      const res = await fetch("/api/guestbook", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name: name.trim(), message: message.trim() }),
      });

      if (!res.ok) throw new Error("Lỗi khi gửi lời nhắn");

      // Reset form và tải lại danh sách
      setName("");
      setMessage("");
      await fetchEntries();
    } catch (err) {
      alert("Không thể gửi lời nhắn. Vui lòng thử lại.");
    } finally {
      setSubmitting(false);
    }
  }

  // Xử lý xóa lời nhắn
  async function handleDelete(id: string) {
    if (!confirm("Bạn có chắc muốn xóa lời nhắn này?")) return;

    try {
      const res = await fetch(`/api/guestbook/${id}`, {
        method: "DELETE",
      });

      if (!res.ok) throw new Error("Lỗi khi xóa");

      await fetchEntries();
```

```
    } catch (err) {
      alert("Không thể xóa lời nhắn. Vui lòng thử lại.");
    }
  }

  return (
    <div className="max-w-3xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-2">Sổ lưu bút</h1>
      <p className="text-gray-500 mb-8">
        Hãy để lại lời nhắn cho tôi nhé!
      </p>

      {/* Form gửi lời nhắn */}
      <form
        onSubmit={handleSubmit}
        className="bg-gray-50 rounded-lg p-6 mb-8 space-y-4"
      >
        <div>
          <label
            htmlFor="name"
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            Tên của bạn
          </label>
          <input
            id="name"
            type="text"
            value={name}
            onChange={(e) => setName(e.target.value)}
            placeholder="Nhập tên của bạn"
            required
            className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
          />
        </div>

        <div>
          <label
            htmlFor="message"
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            Lời nhắn
          </label>
          <textarea
            id="message"
            value={message}
            onChange={(e) => setMessage(e.target.value)}
            placeholder="Viết lời nhắn của bạn..."
            required
            rows={3}
            className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500 resize-none"
          />
        </div>

        <button
          type="submit"
          disabled={submitting || !name.trim() || !message.trim()}
```

```
          className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700
transition-colors disabled:opacity-50 disabled:cursor-not-allowed"
        >
          {submitting ? "Đang gửi..." : "Gửi lời nhắn"}
        </button>
      </form>

      {/* Danh sách lời nhắn */}
      {loading && (
        <div className="text-center py-8 text-gray-500">
          Đang tải sổ lưu bút...
        </div>
      )}

      {error && (
        <div className="text-center py-8 text-red-500">{error}</div>
      )}

      {!loading && !error && (
        <div className="space-y-4">
          <p className="text-sm text-gray-400">
            {entries.length} lời nhắn
          </p>

          {entries.map((entry) => (
            <div
              key={entry.id}
              className="border rounded-lg p-4 hover:shadow-sm transition-shadow"
            >
              <div className="flex items-center justify-between mb-2">
                <span className="font-semibold text-gray-800">
                  {entry.name}
                </span>
                <div className="flex items-center gap-3">
                  <span className="text-xs text-gray-400">
                    {new Date(entry.createdAt).toLocaleDateString("vi-VN")}
                  </span>
                  <button
                    onClick={() => handleDelete(entry.id)}
                    className="text-xs text-red-400 hover:text-red-600 transition-
colors"
                  >
                    Xóa
                  </button>
                </div>
              </div>
              <p className="text-gray-600">{entry.message}</p>
            </div>
          ))}

          {entries.length === 0 && (
            <p className="text-center text-gray-400 py-8">
              Chưa có lời nhắn nào. Hãy là người đầu tiên!
            </p>
          )}
        </div>
      )}
    </div>
```

```
  );
}
```

Giải thích: Giải thích chi tiết:

- `"use client"` — Đánh dấu đây là Client Component vì ta dùng hooks (`useState`, `useEffect`) và event handlers (`onClick`, `onSubmit`).
- `useEffect(() => { fetchEntries(); }, [])` — Gọi API khi component mount lần đầu (tương tự `componentDidMount`).
- `await fetchEntries()` sau mỗi thao tác (thêm/xóa) — Tải lại dữ liệu để cập nhật giao diện.
- `e.preventDefault()` — Ngăn browser reload trang khi submit form.
- `confirm(...)` — Hiện hộp thoại xác nhận trước khi xóa.

Chạy dev server và kiểm tra:

```bash
npm run dev
```

Kết quả mong đợi: Kết quả mong đợi:

- Truy cập `http://localhost:3000/guestbook` — hiển thị 3 lời nhắn mẫu
- Điền form và nhấn "Gửi lời nhắn" — lời nhắn mới xuất hiện ở đầu danh sách
- Nhấn "Xóa" trên lời nhắn — lời nhắn bị xóa sau khi xác nhận

3.3. Client-side fetch: Hạn chế và cải tiến

Cách dùng `useEffect` + `fetch` + `useState` ở trên hoạt động tốt nhưng có một số hạn chế:

Hạn chế	Mô tả
Boilerplate code	Cần quản lý loading, error, data bằng tay
Không có cache	Mỗi lần mount component đều gọi API mới
Không có retry	Nếu API lỗi, phải tự xử lý retry
Không có deduplication	Nhiều component gọi cùng API sẽ tạo nhiều request

Thư viện **SWR** (của Vercel) và **React Query** (TanStack Query) giải quyết tất cả các vấn đề trên. Ví dụ với SWR:

```tsx
// Cài đặt: npm install swr
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((res) => res.json());

function GuestbookPage() {
  const { data, error, isLoading, mutate } = useSWR("/api/guestbook", fetcher);

  // data = danh sách entries (tự cache)
  // mutate() = tải lại data
  // isLoading, error = tự quản lý
}
```

Mẹo: SWR và React Query là các thư viện nâng cao, rất phổ biến trong dự án thực tế. Bạn có thể tìm hiểu thêm tại: https://swr.vercel.app (https://swr.vercel.app)

3.4. Commit thay đổi

```bash
git add .
git commit -m "feat: tạo trang Guestbook với client-side data fetching"
```

Bài tập tự làm — Phần 3

1. Thêm tính năng "loading" cho từng nút Xóa (khi đang xóa, nút hiển thị "Đang xóa..." và bị disable)
2. Cài đặt thư viện SWR (`npm install swr`) và thay thế `useEffect` + `useState` bằng hook `useSWR`. So sánh lượng code trước và sau
3. Thêm tính năng phân trang: hiển thị 5 lời nhắn mỗi trang, có nút "Trang trước" và "Trang sau"

Phần 4: Server Actions & Form Handling

Thời lượng: *Thời lượng: ~35 phút*

Server Actions là cách hiện đại để xử lý form trong Next.JS. Thay vì tạo API route rồi gọi `fetch()` từ client, ta có thể gọi trực tiếp hàm server từ form — đơn giản, type-safe, và hỗ trợ progressive enhancement (form vẫn hoạt động khi JavaScript bị tắt).

4.1. Server Actions là gì?

Server Actions là các hàm async chạy trên server, được đánh dấu bằng directive `"use server"`. Chúng có thể được gọi trực tiếp từ form hoặc từ Client Component.

So sánh hai cách xử lý form:

	Cách truyền thống (API Route)	Server Actions
Bước 1	Tạo file `api/endpoint/route.ts`	Tạo function với `"use server"`
Bước 2	Client gọi `fetch('/api/endpoint')`	Form gọi trực tiếp function
Type-safe	Không (phải serialize/deserialize JSON)	Có (TypeScript end-to-end)
Progressive Enhancement	Không (cần JavaScript)	Có (form hoạt động không cần JS)
Revalidation	Phải tự xử lý	Tích hợp sẵn `revalidatePath()`

4.2. Cài đặt Zod — Schema Validation

Zod là thư viện validation cho TypeScript. Nó giúp định nghĩa schema (cấu trúc dữ liệu) và validate dữ liệu đầu vào một cách type-safe.

Cài đặt Zod:

```bash
npm install zod
```

Ví dụ cơ bản về Zod:

```tsx
import { z } from "zod";

// Định nghĩa schema
const userSchema = z.object({
  name: z.string().min(2, "Tên phải có ít nhất 2 ký tự"),
  email: z.string().email("Email không hợp lệ"),
  age: z.number().min(18, "Phải từ 18 tuổi trở lên"),
});

// Validate dữ liệu
const result = userSchema.safeParse({
  name: "A",
  email: "invalid",
```

```
  age: 15,
});

if (!result.success) {
  console.log(result.error.flatten().fieldErrors);
  // { name: ["Tên phải có ít nhất 2 ký tự"], email: ["Email không hợp lệ"], age:
["Phải từ 18 tuổi trở lên"] }
}
```

Giải thích: Tại sao dùng Zod thay vì validate bằng if/else?

- **Type-safe**: Zod có thể suy ra (infer) TypeScript types từ schema — viết 1 lần, dùng cho cả validation lẫn type checking.
- **Thông báo lỗi rõ ràng**: Mỗi field có message riêng, dễ hiển thị cho người dùng.
- **Composable**: Có thể kết hợp, mở rộng schema dễ dàng.

4.3. Thực hành: Server Action cho form Guestbook

Tạo file `src/app/guestbook/actions.ts`:

```Tsx
"use server";

import { z } from "zod";
import { revalidatePath } from "next/cache";
import { guestbookEntries } from "@/data/guestbook";

// Định nghĩa schema validation cho guestbook
const guestbookSchema = z.object({
  name: z
    .string()
    .min(2, "Tên phải có ít nhất 2 ký tự")
    .max(50, "Tên không được quá 50 ký tự"),
  message: z
    .string()
    .min(1, "Lời nhắn không được để trống")
    .max(500, "Lời nhắn không được quá 500 ký tự"),
});

export interface ActionState {
  success: boolean;
  errors?: {
    name?: string[];
    message?: string[];
  };
}

export async function createGuestbookEntry(
  prevState: ActionState,
  formData: FormData
): Promise<ActionState> {
```

```
  // Lấy dữ liệu từ form
  const rawData = {
    name: formData.get("name") as string,
    message: formData.get("message") as string,
  };

  // Validate bằng Zod
  const result = guestbookSchema.safeParse(rawData);

  if (!result.success) {
    return {
      success: false,
      errors: result.error.flatten().fieldErrors,
    };
  }

  // Thêm entry mới vào mảng
  const newEntry = {
    id: Date.now().toString(),
    name: result.data.name,
    message: result.data.message,
    createdAt: new Date().toISOString(),
  };

  guestbookEntries.unshift(newEntry);

  // Revalidate trang guestbook để hiển thị data mới
  revalidatePath("/guestbook");

  return { success: true };
}

export async function deleteGuestbookEntry(id: string): Promise<ActionState> {
  const index = guestbookEntries.findIndex((entry) => entry.id === id);

  if (index === -1) {
    return { success: false, errors: { message: ["Không tìm thấy lời nhắn"] } };
  }

  guestbookEntries.splice(index, 1);
  revalidatePath("/guestbook");

  return { success: true };
}
```

Giải thích: Giải thích chi tiết:

- `"use server"` — Đánh dấu toàn bộ file chứa Server Actions. Các hàm export từ file này sẽ chạy trên server.
- `createGuestbookEntry(prevState, formData)` — Server Action nhận 2 tham số: `prevState` (trạng thái trước đó, dùng với `useActionState`) và `formData` (dữ liệu form).

- `guestbookSchema.safeParse(rawData)` — Validate dữ liệu. `safeParse` trả về `{ success: true, data }` hoặc `{ success: false, error }` thay vì throw exception.
- `revalidatePath("/guestbook")` — Thông báo Next.JS tải lại dữ liệu của trang `/guestbook`.

4.4. Thực hành: Tạo form component với Server Actions

Bây giờ ta sẽ tạo một phiên bản guestbook sử dụng Server Actions thay vì client-side fetch. Cập nhật trang guestbook để kết hợp cả hai cách tiếp cận.

Tạo file `src/components/guestbook-form.tsx`:

```tsx
"use client";

import { useActionState } from "react";
import { createGuestbookEntry, ActionState } from "@/app/guestbook/actions";

const initialState: ActionState = {
  success: false,
};

export default function GuestbookForm() {
  const [state, formAction, isPending] = useActionState(
    createGuestbookEntry,
    initialState
  );

  return (
    <form action={formAction} className="bg-gray-50 rounded-lg p-6 mb-8 space-y-
4">
      <div>
        <label
          htmlFor="name"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          Tên của bạn
        </label>
        <input
          id="name"
          name="name"
          type="text"
          placeholder="Nhập tên của bạn"
          required
          className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
        />
        {state.errors?.name && (
          <p className="text-red-500 text-sm mt-1">{state.errors.name[0]}</p>
        )}
      </div>
```

```
      <div>
        <label
          htmlFor="message"
          className="block text-sm font-medium text-gray-700 mb-1"
        >
          Lời nhắn
        </label>
        <textarea
          id="message"
          name="message"
          placeholder="Viết lời nhắn của bạn..."
          required
          rows={3}
          className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500 resize-none"
        />
        {state.errors?.message && (
          <p className="text-red-500 text-sm mt-1">
            {state.errors.message[0]}
          </p>
        )}
      </div>

      <button
        type="submit"
        disabled={isPending}
        className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700
transition-colors disabled:opacity-50 disabled:cursor-not-allowed"
      >
        {isPending ? "Đang gửi..." : "Gửi lời nhắn"}
      </button>

      {state.success && (
        <p className="text-green-600 text-sm">Gửi lời nhắn thành công!</p>
      )}
    </form>
  );
}
```

Giải thích: Giải thích:

- `useActionState(createGuestbookEntry, initialState)` — Hook của React 19 dùng để kết nối Server Action với form. Trả về:
  - `state` — Trạng thái hiện tại (kết quả từ Server Action)
  - `formAction` — Function truyền vào `<form action={...}>`
  - `isPending` — `true` khi đang submit
- `<form action={formAction}>` — Thay vì `onSubmit`, ta truyền Server Action vào attribute `action`. Form sẽ tự động gửi dữ liệu đến server.

- `name="name"` và `name="message"` — Attribute `name` trên input rất quan trọng, vì `formData.get("name")` dựa vào attribute này để lấy giá trị.
- `state.errors?.name` — Hiển thị lỗi validation nếu có.

Bây giờ cập nhật trang guestbook để sử dụng GuestbookForm component. Cập nhật file `src/app/guestbook/page.tsx`:

```tsx
import { guestbookEntries } from "@/data/guestbook";
import GuestbookForm from "@/components/guestbook-form";
import DeleteButton from "@/components/delete-button";

export default function GuestbookPage() {
  const entries = guestbookEntries;

  return (
    <div className="max-w-3xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-2">Sổ lưu bút</h1>
      <p className="text-gray-500 mb-8">Hãy để lại lời nhắn cho tôi nhé!</p>

      <GuestbookForm />

      <div className="space-y-4">
        <p className="text-sm text-gray-400">{entries.length} lời nhắn</p>

        {entries.map((entry) => (
          <div
            key={entry.id}
            className="border rounded-lg p-4 hover:shadow-sm transition-shadow"
          >
            <div className="flex items-center justify-between mb-2">
              <span className="font-semibold text-gray-800">{entry.name}</span>
              <div className="flex items-center gap-3">
                <span className="text-xs text-gray-400">
                  {new Date(entry.createdAt).toLocaleDateString("vi-VN")}
                </span>
                <DeleteButton id={entry.id} />
              </div>
            </div>
            <p className="text-gray-600">{entry.message}</p>
          </div>
        ))}

        {entries.length === 0 && (
          <p className="text-center text-gray-400 py-8">
            Chưa có lời nhắn nào. Hãy là người đầu tiên!
          </p>
        )}
      </div>
    </div>
  );
}
```

Giải thích: Chú ý sự khác biệt quan trọng: Trang này **không còn `"use client"`** nữa! Đây giờ là **Server Component** — truy cập dữ liệu trực tiếp từ `guestbookEntries` mà không cần gọi API. Form được tách thành Client Component riêng (`GuestbookForm`).

Tạo file `src/components/delete-button.tsx`:

```tsx
"use client";

import { deleteGuestbookEntry } from "@/app/guestbook/actions";

export default function DeleteButton({ id }: { id: string }) {
  async function handleDelete() {
    if (!confirm("Bạn có chắc muốn xóa lời nhắn này?")) return;
    await deleteGuestbookEntry(id);
  }

  return (
    <button
      onClick={handleDelete}
      className="text-xs text-red-400 hover:text-red-600 transition-colors"
    >
      Xóa
    </button>
  );
}
```

Chạy dev server và kiểm tra:

```bash
npm run dev
```

Kết quả mong đợi: Kết quả mong đợi:

- Trang Guestbook hiển thị danh sách lời nhắn (Server Component — tải nhanh hơn)
- Điền form và submit — lời nhắn mới xuất hiện ngay, form tự reset (nhờ Server Action + revalidatePath)
- Thử submit form với tên chỉ 1 ký tự — hiển thị lỗi validation "Tên phải có ít nhất 2 ký tự"
- Nhấn Xóa — lời nhắn bị xóa

Mẹo: So sánh: Phiên bản Server Actions này đơn giản hơn phiên bản client-side fetch ở Phần 3 — ít state cần quản lý, validation tự động, và hỗ trợ progressive enhancement.

4.5. Thực hành: Cải tiến form Liên hệ

Bây giờ ta áp dụng Server Actions cho trang Contact — biến trang thông tin tĩnh thành form liên hệ thực sự.

Tạo file `src/app/contact/actions.ts`:

```tsx
"use server";

import { z } from "zod";

const contactSchema = z.object({
  name: z
    .string()
    .min(2, "Tên phải có ít nhất 2 ký tự")
    .max(100, "Tên không được quá 100 ký tự"),
  email: z
    .string()
    .email("Email không hợp lệ"),
  subject: z
    .string()
    .min(5, "Tiêu đề phải có ít nhất 5 ký tự")
    .max(200, "Tiêu đề không được quá 200 ký tự"),
  message: z
    .string()
    .min(10, "Nội dung phải có ít nhất 10 ký tự")
    .max(2000, "Nội dung không được quá 2000 ký tự"),
});

export interface ContactFormState {
  success: boolean;
  errors?: {
    name?: string[];
    email?: string[];
    subject?: string[];
    message?: string[];
  };
}

export async function sendContactMessage(
  prevState: ContactFormState,
  formData: FormData
): Promise<ContactFormState> {
  const rawData = {
    name: formData.get("name") as string,
    email: formData.get("email") as string,
    subject: formData.get("subject") as string,
    message: formData.get("message") as string,
  };
```

```
  const result = contactSchema.safeParse(rawData);

  if (!result.success) {
    return {
      success: false,
      errors: result.error.flatten().fieldErrors,
    };
  }

  // Trong thực tế, ở đây sẽ gửi email hoặc lưu vào database
  // Hiện tại ta chỉ log ra console (server-side)
  console.log("Tin nhắn liên hệ mới:", result.data);

  return { success: true };
}
```

Cập nhật file `src/app/contact/page.tsx`:

```
Tsx
"use client";

import { useActionState } from "react";
import { sendContactMessage, ContactFormState } from "./actions";

const initialState: ContactFormState = {
  success: false,
};

export default function ContactPage() {
  const [state, formAction, isPending] = useActionState(
    sendContactMessage,
    initialState
  );

  return (
    <div className="max-w-3xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-2">Liên hệ</h1>
      <p className="text-gray-500 mb-8">
        Bạn có câu hỏi hoặc muốn hợp tác? Hãy gửi tin nhắn cho tôi!
      </p>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
        {/* Thông tin liên hệ */}
        <div className="space-y-4">
          <div className="bg-gray-50 rounded-lg p-4">
            <h3 className="font-semibold mb-1">Email</h3>
            <a
              href="mailto:nguyenvana@sv.dlu.edu.vn"
              className="text-blue-600 hover:underline text-sm"
            >
              nguyenvana@sv.dlu.edu.vn
            </a>
          </div>
          <div className="bg-gray-50 rounded-lg p-4">
            <h3 className="font-semibold mb-1">GitHub</h3>
```

```
            <a
              href="https://github.com/nguyenvana"
              target="_blank"
              rel="noopener noreferrer"
              className="text-blue-600 hover:underline text-sm"
            >
              github.com/nguyenvana
            </a>
          </div>
          <div className="bg-gray-50 rounded-lg p-4">
            <h3 className="font-semibold mb-1">Địa chỉ</h3>
            <p className="text-sm text-gray-600">
              Đại học Đà Lạt, 01 Phù Đổng Thiên Vương, Đà Lạt
            </p>
          </div>
        </div>

        {/* Form liên hệ */}
        <div className="md:col-span-2">
          {state.success ? (
            <div className="bg-green-50 border border-green-200 rounded-lg p-6
text-center">
              <h3 className="text-green-700 font-semibold text-lg mb-2">
                Gửi thành công!
              </h3>
              <p className="text-green-600">
                Cảm ơn bạn đã liên hệ. Tôi sẽ phản hồi sớm nhất có thể.
              </p>
            </div>
          ) : (
            <form action={formAction} className="space-y-4">
              <div>
                <label
                  htmlFor="name"
                  className="block text-sm font-medium text-gray-700 mb-1"
                >
                  Họ và tên
                </label>
                <input
                  id="name"
                  name="name"
                  type="text"
                  placeholder="Nguyễn Văn A"
                  required
                  className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
                />
                {state.errors?.name && (
                  <p className="text-red-500 text-sm mt-1">
                    {state.errors.name[0]}
                  </p>
                )}
              </div>

              <div>
                <label
                  htmlFor="email"
                  className="block text-sm font-medium text-gray-700 mb-1"
```

```
                >
                  Email
                </label>
                <input
                  id="email"
                  name="email"
                  type="email"
                  placeholder="email@example.com"
                  required
                  className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
                />
                {state.errors?.email && (
                  <p className="text-red-500 text-sm mt-1">
                    {state.errors.email[0]}
                  </p>
                )}
              </div>

              <div>
                <label
                  htmlFor="subject"
                  className="block text-sm font-medium text-gray-700 mb-1"
                >
                  Tiêu đề
                </label>
                <input
                  id="subject"
                  name="subject"
                  type="text"
                  placeholder="Chủ đề bạn muốn trao đổi"
                  required
                  className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
                />
                {state.errors?.subject && (
                  <p className="text-red-500 text-sm mt-1">
                    {state.errors.subject[0]}
                  </p>
                )}
              </div>

              <div>
                <label
                  htmlFor="message"
                  className="block text-sm font-medium text-gray-700 mb-1"
                >
                  Nội dung
                </label>
                <textarea
                  id="message"
                  name="message"
                  placeholder="Viết nội dung tin nhắn..."
                  required
                  rows={5}
                  className="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500 resize-none"
                />
```

```
                {state.errors?.message && (
                  <p className="text-red-500 text-sm mt-1">
                    {state.errors.message[0]}
                  </p>
                )}
              </div>

              <button
                type="submit"
                disabled={isPending}
                className="w-full bg-blue-600 text-white px-6 py-3 rounded-lg
hover:bg-blue-700 transition-colors disabled:opacity-50 disabled:cursor-not-
allowed"
              >
                {isPending ? "Đang gửi..." : "Gửi tin nhắn"}
              </button>
            </form>
          )}
        </div>
      </div>
    </div>
  );
}
```

Kết quả mong đợi: Kết quả mong đợi:

- Trang Contact bây giờ có form liên hệ với 4 trường: Họ tên, Email, Tiêu đề, Nội dung
- Submit form thiếu thông tin → hiển thị validation errors từ Zod
- Submit form thành công → hiển thị thông báo "Gửi thành công!"
- Kiểm tra terminal server → thấy log "Tin nhắn liên hệ mới: ..."

4.6. Commit thay đổi

```bash
git add .
git commit -m "feat: thêm Server Actions cho Guestbook và Contact form với Zod
validation"
```

Bài tập tự làm — Phần 4

1. Thêm tính năng "reset form" — sau khi gửi thành công ở trang Contact, thêm nút "Gửi tin nhắn khác" để quay lại form
2. Thêm validation cho trang Guestbook: không cho phép gửi lời nhắn trùng lặp (cùng name + message) trong vòng 1 phút

3. Tìm hiểu thêm về `useFormStatus()` hook — sử dụng nó để tạo button submit component tái sử dụng được cho nhiều form

Phần 5: shadcn/ui — Component Library

Thời lượng: *Thời lượng: ~35 phút*

Trong phần này, chúng ta sẽ cài đặt và sử dụng **shadcn/ui** — một component library hiện đại, để nâng cấp giao diện website từ Tailwind CSS thuần lên một tầm mới chuyên nghiệp hơn.

5.1. shadcn/ui là gì?

shadcn/ui **không phải** là một package npm thông thường. Đây là bộ sưu tập các component được thiết kế sẵn, khi cài đặt sẽ **copy trực tiếp source code** vào project của bạn.

So sánh với các component library khác:

Đặc điểm	Material UI / Ant Design	shadcn/ui
Cài đặt	`npm install @mui/material`	`npx shadcn@latest add button`
Code nằm ở	`node_modules/`	`src/components/ui/`
Tùy chỉnh	Override styles (phức tạp)	Sửa trực tiếp source code
Bundle size	Lớn (import cả library)	Nhỏ (chỉ có component bạn dùng)
Nền tảng	Riêng biệt	Radix UI + Tailwind CSS

Mẹo: Ưu điểm lớn nhất: Bạn **sở hữu** code — có thể đọc, hiểu, và sửa đổi bất kỳ component nào theo ý muốn. Không bị phụ thuộc vào bản cập nhật library.

5.2. Cài đặt shadcn/ui

Trong project `ctk46-lab01`, chạy lệnh khởi tạo:

```bash
npx shadcn@latest init
```

Khi được hỏi, chọn các tùy chọn sau:

```
Which color would you like to use as the base color? › Neutral
```

```
Do you want to use CSS variables for theming? › yes
```

Lưu ý: Lưu ý: Lệnh init sẽ tự động cập nhật một số file cấu hình (`tailwind.config.ts`, `globals.css`, `tsconfig.json`). Đây là hành vi bình thường.

Kết quả mong đợi: Sau khi cài đặt, bạn sẽ thấy:

- File `components.json` mới ở thư mục gốc — file cấu hình shadcn/ui
- Thư mục `src/components/ui/` — nơi chứa các component khi cài đặt
- File `src/lib/utils.ts` — hàm tiện ích `cn()` để merge Tailwind classes

Kiểm tra file `src/lib/utils.ts`:

```tsx
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

Giải thích: Giải thích: Hàm `cn()` kết hợp `clsx` (conditional classes) và `twMerge` (xử lý xung đột Tailwind classes). Ví dụ: `cn("px-4 py-2", isActive && "bg-blue-500", "px-6")` → kết quả là `"py-2 px-6 bg-blue-500"` (px-6 override px-4).

5.3. Cài đặt các components cần thiết

Cài đặt các component ta sẽ dùng:

```bash
npx shadcn@latest add button card input textarea label badge separator
```

Kết quả mong đợi: Kết quả: Các file component được tạo trong `src/components/ui/`:

```
src/components/ui/
├── button.tsx
├── card.tsx
├── input.tsx
├── textarea.tsx
├── label.tsx
├── badge.tsx
└── separator.tsx
```

Mẹo: Mỗi component là 1 file TypeScript hoàn chỉnh. Bạn có thể mở đọc và sửa đổi tùy ý.

5.4. Sử dụng Button component

Mở file `src/components/ui/button.tsx` để xem cấu trúc. Bạn sẽ thấy component hỗ trợ nhiều **variant** (kiểu hiển thị) và **size** (kích thước).

Cách sử dụng:

```tsx
import { Button } from "@/components/ui/button";

// Variants
<Button>Default</Button>
<Button variant="destructive">Xóa</Button>
<Button variant="outline">Outline</Button>
<Button variant="secondary">Secondary</Button>
<Button variant="ghost">Ghost</Button>
<Button variant="link">Link</Button>

// Sizes
<Button size="sm">Nhỏ</Button>
<Button size="default">Mặc định</Button>
<Button size="lg">Lớn</Button>

// Disabled + Loading
<Button disabled>Disabled</Button>
```

5.5. Sử dụng Card component

```tsx
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@/components/ui/card";

<Card>
  <CardHeader>
    <CardTitle>Tiêu đề</CardTitle>
    <CardDescription>Mô tả ngắn</CardDescription>
  </CardHeader>
  <CardContent>
    <p>Nội dung chính</p>
  </CardContent>
  <CardFooter>
    <Button>Hành động</Button>
  </CardFooter>
```

```
</Card>
```

5.6. Thực hành: Nâng cấp trang Blog với shadcn/ui

Cập nhật file `src/app/blog/page.tsx` để sử dụng Card và Badge:

```tsx
import Link from "next/link";
import { Post } from "@/types/post";
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";

async function getPosts(): Promise<Post[]> {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    next: { revalidate: 60 },
  });

  if (!res.ok) {
    throw new Error("Không thể tải danh sách bài viết");
  }

  return res.json();
}

export default async function BlogPage() {
  const posts = await getPosts();

  return (
    <div>
      <h1 className="text-3xl font-bold mb-2">Blog</h1>
      <p className="text-gray-500 mb-6">
        Tổng cộng {posts.length} bài viết từ API
      </p>

      <div className="space-y-4">
        {posts.slice(0, 10).map((post) => (
          <Link key={post.id} href={`/blog/${post.id}`}>
            <Card className="hover:shadow-md transition-shadow cursor-pointer">
              <CardHeader>
                <div className="flex items-center gap-2 mb-1">
                  <Badge variant="secondary">Tác giả #{post.userId}</Badge>
                  <span className="text-sm text-gray-400">
                    Bài #{post.id}
                  </span>
                </div>
                <CardTitle className="capitalize">{post.title}</CardTitle>
                <CardDescription className="line-clamp-2">
                  {post.body}
                </CardDescription>
```

```
              </CardHeader>
            </Card>
          </Link>
        ))}
      </div>
    </div>
  );
}
```

5.7. Thực hành: Nâng cấp form Guestbook với shadcn/ui

Cập nhật file `src/components/guestbook-form.tsx`:

```tsx
"use client";

import { useActionState } from "react";
import { createGuestbookEntry, ActionState } from "@/app/guestbook/actions";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";

const initialState: ActionState = {
  success: false,
};

export default function GuestbookForm() {
  const [state, formAction, isPending] = useActionState(
    createGuestbookEntry,
    initialState
  );

  return (
    <Card className="mb-8">
      <CardHeader>
        <CardTitle>Viết lời nhắn</CardTitle>
      </CardHeader>
      <CardContent>
        <form action={formAction} className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="name">Tên của bạn</Label>
            <Input
              id="name"
              name="name"
              placeholder="Nhập tên của bạn"
              required
            />
            {state.errors?.name && (
              <p className="text-red-500 text-sm">{state.errors.name[0]}</p>
            )}
          </div>

          <div className="space-y-2">
```

```
            <Label htmlFor="message">Lời nhắn</Label>
            <Textarea
              id="message"
              name="message"
              placeholder="Viết lời nhắn của bạn..."
              required
              rows={3}
            />
            {state.errors?.message && (
              <p className="text-red-500 text-sm">
                {state.errors.message[0]}
              </p>
            )}
          </div>

          <Button type="submit" disabled={isPending}>
            {isPending ? "Đang gửi..." : "Gửi lời nhắn"}
          </Button>

          {state.success && (
            <p className="text-green-600 text-sm">
              Gửi lời nhắn thành công!
            </p>
          )}
        </form>
      </CardContent>
    </Card>
  );
}
```

Cập nhật file `src/app/guestbook/page.tsx` để dùng Card cho danh sách lời nhắn:

```tsx
import { guestbookEntries } from "@/data/guestbook";
import GuestbookForm from "@/components/guestbook-form";
import DeleteButton from "@/components/delete-button";
import { Card, CardContent } from "@/components/ui/card";
import { Separator } from "@/components/ui/separator";

export default function GuestbookPage() {
  const entries = guestbookEntries;

  return (
    <div className="max-w-3xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-2">Sổ lưu bút</h1>
      <p className="text-gray-500 mb-8">Hãy để lại lời nhắn cho tôi nhé!</p>

      <GuestbookForm />

      <Separator className="my-8" />

      <div className="space-y-4">
        <p className="text-sm text-gray-400">{entries.length} lời nhắn</p>

        {entries.map((entry) => (
          <Card key={entry.id}>
```

```
            <CardContent className="pt-4">
              <div className="flex items-center justify-between mb-2">
                <span className="font-semibold text-gray-800">
                  {entry.name}
                </span>
                <div className="flex items-center gap-3">
                  <span className="text-xs text-gray-400">
                    {new Date(entry.createdAt).toLocaleDateString("vi-VN")}
                  </span>
                  <DeleteButton id={entry.id} />
                </div>
              </div>
              <p className="text-gray-600">{entry.message}</p>
            </CardContent>
          </Card>
        ))}

        {entries.length === 0 && (
          <p className="text-center text-gray-400 py-8">
            Chưa có lời nhắn nào. Hãy là người đầu tiên!
          </p>
        )}
      </div>
    </div>
  );
}
```

5.8. Thực hành: Nâng cấp trang Projects với Card và Badge

Cập nhật file `src/app/projects/page.tsx`:

```
Tsx
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";

const projects = [
  {
    title: "Website Portfolio",
    description: "Website cá nhân xây dựng bằng Next.JS và Tailwind CSS",
    tech: ["Next.JS", "Tailwind CSS", "TypeScript"],
    status: "Đang phát triển",
  },
  {
    title: "Ứng dụng Quản lý Công việc",
    description: "Ứng dụng Todo App với React và Local Storage",
    tech: ["React", "CSS Modules", "JavaScript"],
    status: "Hoàn thành",
  },
  {
```

```
    title: "API RESTful",
    description: "API quản lý sản phẩm với Node.js và Express",
    tech: ["Node.js", "Express", "MongoDB"],
    status: "Hoàn thành",
  },
  {
    title: "Chat Realtime",
    description: "Ứng dụng chat realtime với Socket.IO",
    tech: ["React", "Socket.IO", "Node.js"],
    status: "Đang phát triển",
  },
];

export default function ProjectsPage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Dự án</h1>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        {projects.map((project, index) => (
          <Card key={index} className="hover:shadow-md transition-shadow">
            <CardHeader>
              <div className="flex items-center justify-between">
                <CardTitle className="text-lg">{project.title}</CardTitle>
                <Badge
                  variant={
                    project.status === "Hoàn thành" ? "default" : "secondary"
                  }
                >
                  {project.status}
                </Badge>
              </div>
              <CardDescription>{project.description}</CardDescription>
            </CardHeader>
            <CardContent>
              <div className="flex flex-wrap gap-2">
                {project.tech.map((t) => (
                  <Badge key={t} variant="outline">
                    {t}
                  </Badge>
                ))}
              </div>
            </CardContent>
          </Card>
        ))}
      </div>
    </div>
  );
}
```

5.9. Kiểm tra kết quả

Chạy dev server:

```
Bash
```

```
npm run dev
```

Kiểm tra từng trang:

Trang	URL	Kiểm tra
Blog	`/blog`	Card component cho bài viết, Badge cho tác giả
Guestbook	`/guestbook`	Card form, Input/Textarea/Button từ shadcn/ui
Projects	`/projects`	Card + Badge cho project, status badge
Contact	`/contact`	Form liên hệ hoạt động

Kết quả mong đợi: Kết quả mong đợi: Toàn bộ giao diện trông nhất quán và chuyên nghiệp hơn, với các component shadcn/ui có style đồng bộ (bo góc, shadow, spacing).

5.10. Commit thay đổi

```Bash
git add .
git commit -m "feat: cài đặt shadcn/ui và nâng cấp UI cho Blog, Guestbook,
Projects"
```

Bài tập tự làm — Phần 5

1. Cài thêm component **Dialog** (`npx shadcn@latest add dialog`) và sử dụng để hiện form xác nhận trước khi xóa lời nhắn (thay vì `confirm()`)
2. Cài thêm component **Avatar** và sử dụng cho trang About và cho mỗi entry guestbook (hiển thị chữ cái đầu tên)
3. Nâng cấp trang Contact sử dụng Input, Textarea, Button, Label từ shadcn/ui thay vì HTML element thuần
4. Khám phá thêm các component tại https://ui.shadcn.com/docs/components (https://ui.shadcn.com/docs/components) và thử cài đặt, sử dụng 2-3 component mới

Phần 6: Tổng kết & Yêu cầu nộp bài

Thời lượng: *Thời lượng: ~10 phút*

Tóm tắt kiến thức

Trong bài thực hành này, bạn đã học và thực hành:

Chủ đề	Kiến thức chính
Server-side Fetch	`fetch()` trong Server Component, async components, caching, revalidation
API Routes	Route Handlers (`route.ts`), GET/POST/DELETE, NextRequest/NextResponse
Client-side Fetch	`useEffect` + `fetch` + `useState`, khi nào dùng client vs server fetch
Server Actions	`"use server"`, `useActionState`, `revalidatePath`, progressive enhancement
Zod Validation	Schema definition, `safeParse`, field errors, type inference
shadcn/ui	Component library, Button/Card/Input/Badge, cài đặt và tùy chỉnh

Checklist hoàn thành

Trước khi nộp bài, hãy đảm bảo bạn đã hoàn thành tất cả các mục sau:

- ☐ Trang Blog fetch dữ liệu từ JSONPlaceholder API (không còn dữ liệu hardcoded)
- ☐ Trang chi tiết Blog (`/blog/[id]`) hiển thị bài viết + thông tin tác giả từ API
- ☐ Tạo API Routes cho Guestbook: GET, POST, DELETE hoạt động đúng
- ☐ Trang Guestbook hiển thị danh sách + form gửi lời nhắn
- ☐ Form Guestbook sử dụng Server Actions + Zod validation
- ☐ Trang Contact có form liên hệ với Server Actions + validation
- ☐ Cài đặt shadcn/ui thành công
- ☐ Sử dụng ít nhất 3 components shadcn/ui (Button, Card, Input, ...)
- ☐ Nâng cấp UI trang Blog, Projects, Guestbook với shadcn/ui

☐ Commit đúng chuẩn Conventional Commits

☐ Push toàn bộ code lên GitHub

Bài tập nộp điểm

Yêu cầu: Xây dựng trang hiển thị danh sách + chi tiết với data fetching từ public API thực tế.

Chi tiết:

1. Chọn **1 public API** từ danh sách gợi ý (hoặc API khác):
   - PokéAPI: https://pokeapi.co (https://pokeapi.co) — Dữ liệu Pokémon
   - Rick and Morty API: https://rickandmortyapi.com (https://rickandmortyapi.com) — Nhân vật phim
   - The Movie DB: https://www.themoviedb.org/documentation/api (https://www.themoviedb.org/documentation/api) — Phim
   - REST Countries: https://restcountries.com (https://restcountries.com) — Quốc gia
   - Open Library: https://openlibrary.org/developers/api (https://openlibrary.org/developers/api) — Sách
2. Tạo **trang danh sách** hiển thị dữ liệu dạng grid hoặc list (sử dụng Card component)
3. Tạo **trang chi tiết** với dynamic route (`/[id]` hoặc `/[slug]`) hiển thị thông tin đầy đủ
4. **Yêu cầu kỹ thuật:**
   - Sử dụng Server Component + server-side fetch
   - TypeScript interfaces cho dữ liệu API
   - Xử lý loading state và error handling
   - Sử dụng ít nhất 2 shadcn/ui components
   - Responsive design (hiển thị tốt trên mobile và desktop)

Hình thức nộp: Push code lên GitHub repository, gửi link qua hệ thống quản lý bài tập của lớp.

Chuẩn bị cho buổi sau

Buổi 4 sẽ học về **Supabase — Backend-as-a-Service & Authentication**. Để chuẩn bị tốt, hãy:

1. Đăng ký tài khoản Supabase: https://supabase.com (https://supabase.com)
2. Đọc trước tài liệu Getting Started: https://supabase.com/docs/guides/getting-started (https://supabase.com/docs/guides/getting-started)
3. Ôn lại kiến thức SQL cơ bản: CREATE TABLE, SELECT, INSERT, UPDATE, DELETE
4. Tìm hiểu về khái niệm Authentication (đăng ký, đăng nhập, phiên đăng nhập)

Tài liệu tham khảo

- Next.JS Data Fetching: https://nextjs.org/docs/app/building-your-application/data-fetching (https://nextjs.org/docs/app/building-your-application/data-fetching)
- Next.JS Route Handlers: https://nextjs.org/docs/app/building-your-application/routing/route-handlers (https://nextjs.org/docs/app/building-your-application/routing/route-handlers)
- Next.JS Server Actions: https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations (https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations)
- Next.JS Caching: https://nextjs.org/docs/app/building-your-application/caching (https://nextjs.org/docs/app/building-your-application/caching)
- Zod Documentation: https://zod.dev (https://zod.dev)
- shadcn/ui: https://ui.shadcn.com (https://ui.shadcn.com)
- shadcn/ui Components: https://ui.shadcn.com/docs/components (https://ui.shadcn.com/docs/components)
- JSONPlaceholder: https://jsonplaceholder.typicode.com (https://jsonplaceholder.typicode.com)
- SWR: https://swr.vercel.app (https://swr.vercel.app)

- Tailwind CSS: https://tailwindcss.com/docs (https://tailwindcss.com/docs)